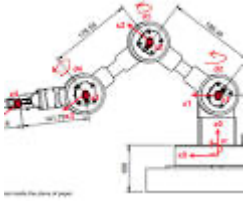
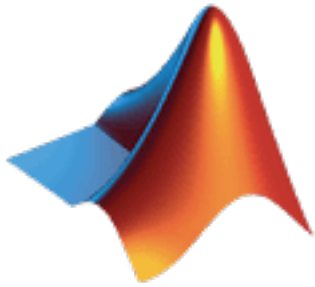


AI Overview



You can simulate a 4-degree-of-freedom (DOF) robot arm in MATLAB using the [Robotic System Toolbox](#) by defining a `rigidBodyTree` model with Denavit-Hartenberg (DH) parameters.



MathWorks +1

Define DH Parameters

Create a standard DH parameter matrix where each row represents a joint `[theta, d, a, alpha]`:

- **theta**: Joint angle (rotation about).
- **d**: Link offset (translation along).
- **a**: Link length (translation along).
- **alpha**: Link twist (rotation about).



YouTube:Chris Annin

For a basic 4-DOF planar or articulated robot (e.g., 3 revolute joints and 1 base/wrist rotation), define your table in MATLAB:

matlab

```
% DH Parameters [theta d a alpha]
dh_params = [
    0,    1.5,    0,    pi/2; % Joint 1
    pi/2, 0,    2.0,    0;   % Joint 2
    0,    0,    1.5,    0;   % Joint 3
    0,    0,    1.0,    0    % Joint 4
];
```

Use code with caution.

Build and Visualize the Robot

Use a `rigidBodyTree` object and loop through the DH parameters to add rigid bodies and joints:

matlab

```
% Initialize robot rigid body tree
robot = rigidBodyTree('DataFormat','row');

% Define joint types (e.g., all revolute 'revolute')
jointType = {'revolute', 'revolute', 'revolute', 'revolute'};

body = cell(4,1);
joint = cell(4,1);

for i = 1:4
    body{i} = rigidBody(['link' num2str(i)]);
    joint{i} = joint(['joint' num2str(i)], jointType{i});

    % Set DH parameters using transform function
    setFixedTransform(joint{i}, dh_params(i,:), 'dh');

    body{i}.Joint = joint{i};

    if i == 1
        addBody(robot, body{i}, robot.BaseName);
    else
        addBody(robot, body{i}, body{i-1}.Name);
    end
end

% Show the 3D visualization
figure;
show(robot);
axis([-3 3 -3 3 0 4]);
title('4-DOF Robot Arm Simulation');
```

Use code with caution.

Simulate Motion

Animate the robot by passing a vector of joint configurations into the `show` function:

matlab

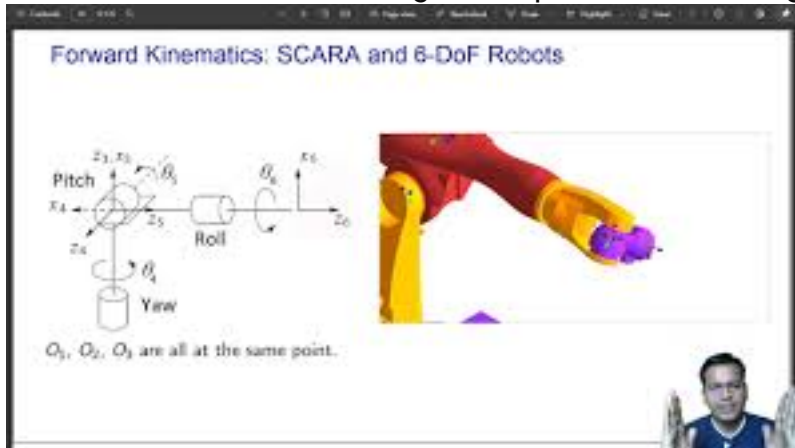
```
% Example joint configuration vector [q1, q2, q3, q4]
q_sample = [pi/4, -pi/6, pi/3, pi/12];

figure;
show(robot, q_sample);
```

```
axis([-3 3 -3 3 0 4]);
```

Use code with caution.

Watch this tutorial on building a manipulator robot using kinematic DH parameters in MATLAB:



20m

Forward Kinematics: Example of 4-DoF SCARA and 6- ...

9.2K views · 4 years ago



YouTube · Arun Dayal Udai

If you'd like, let me know:

- Your specific **DH parameter values** or robot configuration (e.g., SCARA, articulated)
- Whether you want to perform **forward kinematics animation** or **inverse kinematics trajectory planning**

I can tailor the MATLAB script to your exact setup.